Beyond the Terminal: Building VIVA for Vibe Coders

A semester-long journey with Honda Research Institute (99P Labs) from an ambitious codebase knowledge layer to VIVA (Visual Interface for Version Assistance)—a pragmatic desktop app that makes version control safe and accessible.

1. The Difference Between Explaining Code and Saving It

Our team started the semester with a grand vision: the "Code Repo Knowledge Layer." The initial problem statement was that developers jumping into a new codebase struggle to understand the implicit structural and contextual knowledge—the "why was this written this way?" that usually lives only in people's heads or Slack messages. We wanted to build an AI coaching agent, an interactive code map, and an auto-documentation engine all wrapped in a VS Code extension.

We built the Phase 1 deliverables: a VS Code sidebar chat panel, AST-based code parsing using tree-sitter, and a usage tracking layer analyzing Git history via PyDriller. But we quickly hit a wall.

The core bottleneck wasn't the AI. It was the data, the infrastructure, and the user reality. We didn't have access to the rich, proprietary corporate data needed to make a usage tracking tool truly valuable. Furthermore, the "cold start" problem plagued our Q&A bot: it needed senior developers to seed it with answers, but they were already using existing tools like GitHub Copilot.

Then, during a feedback meeting, our 99P Labs mentor, Ryan, posed a question that flipped the project: *"How do we make vibe coding easier? People do not know what they want—AI can give a high-level plan. Think about the vibe coders using phones or tablets to code."*

We realized that while AI tools were democratizing code generation, they were leaving a massive gap in code management. Vibe coders, designers, students, and early-stage developers could generate Python scripts or React components with AI, but they were terrified of the terminal. They didn't need another tool explaining their codebase; they didn't want to learn Git commands. They needed a tool that prevented them from accidentally deleting their own work. We needed to pivot from a web-dependent "Knowledge Layer" to VIVA.

2. Embracing the Mess: Constraints as a Catalyst

Pivoting to a desktop application was an intentional product strategy driven by strict constraints. We lacked external data access and the resources to deploy and maintain a robust web infrastructure (servers, databases, authentication, etc.).
By moving to a local-first Electron app, we eliminated data dependency. We could leverage the user's local project folder and Git history directly. The architecture shifted to a React UI (Renderer Process) talking to a local Git Service (simple-git) and a local SQLite database via a secure Electron IPC bridge. The AI layer (OpenAI/Anthropic) became a modular, opt-in enhancement rather than the core operational dependency.
Our problem statement evolved from "Repository data is hard to analyze and document" to "Beginner developers and AI-assisted coders struggle to safely execute Git/GitHub workflows."

3. Code Loss Prevention over "AI Magic"

If you build an AI tool that handles version control, the most important feature isn't the AI—it's safety. The fastest way to lose a user's trust is to let an automated process force-push over their main branch or silently initialize a repository in the wrong directory.
We designed VIVA with a "Safety-First" philosophy. We explicitly blocked dangerous operations: no silent git init, no default branch deletion, no automatic force pushes, and no automatic pull/merge conflict resolutions. Every destructive or critical action requires explicit user consent.
Instead of exposing the raw terminal, we abstracted Git into a visual workflow:

- **Save Progress:** A one-click local snapshot replacing the manual stage-and-commit dance.
- **Full File Visibility:** Tracked, staged, and untracked files in one clear UI panel.
- **Pull Preview:** Inspecting incoming changes before committing to a merge.

4. The 6 Core AI Components

Rather than making an AI wrapper, we positioned AI as a set of surgical interventions within VIVA. These features only step in when the user encounters a specific friction point, and they all gracefully degrade if the AI API is unavailable.

1. **Auto Commit Message:** The app reads the staged diff and generates a single, plain-language sentence (e.g., "Updated the homepage background color"). It enforces a 3-5 second timeout so saving is never blocked. These generated summaries are stored in the local SQLite DB to power other features.
2. **Natural Language Undo:** Instead of searching for Git SHA hashes, users can ask, "Go back to before I changed the upload flow." The system queries the stored AI commit summaries, finds the closest match, and previews the exact file changes before executing

a rollback.

3. **Untracked File Review:** When new files appear, the app classifies them to either be committed or deleted. We built a two-tier decision engine: a lightning-fast, rule-based pass instantly identifies node_modules or .env files for deletion, saving API tokens. The AI is only invoked for ambiguous files.
4. **File Insight:** A read-only explanation of a selected file's purpose, behavior, and relationship to the project, utilizing recent commit history to provide context.
5. **Smart Conflict Resolver:** When a merge fails, the app surfaces a guided file-by-file dialog without exposing raw Git conflict markers. The AI explains the tradeoff (e.g., "Keep your version—it has the new layout changes") and drafts the merge message once resolved.
6. **Weekly Report:** The app compiles Git statistics (lines added/removed) and the stored AI summaries into a readable narrative of what was accomplished that week.

5. What's Next: Measuring Success by Safety, Not Features

Our journey from the Code Repo Knowledge Layer to VIVA taught us that workflow-first design outlives AI-first design. AI features are easy to build, but integrating them into a process where users feel genuinely safe is much harder.

Moving forward, our roadmap isn't focused on adding more LLM endpoints. It's focused on success metrics that prove our core thesis. We are tracking the time it takes for a beginner to achieve their first commit, the percentage of users who successfully navigate a merge conflict without abandoning the flow, and the reduction in user anxiety regarding code loss.

The future of coding might be driven by natural language generation, but securing that code still requires a reliable ratchet. We built the ratchet.